1. Tensors

Tensors are a specialized data structure that are very similar to arrays and matrices. In PyTorch, we use tensors to encode the inputs and outputs of a model, as well as the model's parameters.

Tensors are similar to NumPy's ndarrays, except that tensors can run on GPUs or other specialized hardware to accelerate computing. If you're familiar with ndarrays, you'll be right at home with the Tensor API. If not, follow along in this quick API walkthrough.

In [1]:
```
import torch
import numpy as np
```

In [2]:
```
print(torch.__version__)
```

```
2.7.0+cpu
```

Tensor Initialization

Tensors can be initialized in various ways. Take a look at the following examples:

Directly from data

Tensors can be created directly from data. The data type is automatically inferred.

In [3]:
```
data = [[1, 2],[3, 4]]  # data is a 2D list
x_data = torch.tensor(data)
print (x_data)
```

```
tensor([[1, 2],
        [3, 4]])
```

From a NumPy array

Tensors can be created from NumPy arrays (and vice versa - see *bridge-to-np-label*).

In [4]:
```
np_array = np.array(data)
x_np = torch.from_numpy(np_array)
print("data:" , data)
print("np_array:" , np_array)
print ("x_np:" , x_np)
```

```
data: [[1, 2], [3, 4]]
np_array: [[1 2]
 [3 4]]
x_np: tensor([[1, 2],
        [3, 4]])
```

From another tensor:

The new tensor retains the properties (shape, datatype) of the argument tensor, unless explicitly overridden.

In [5]:
```
x_ones = torch.ones_like(x_data) # retains the properties of x_data (shape and data
print(f"Ones Tensor: \n {x_ones} \n")

x_rand = torch.rand_like(x_data, dtype=torch.float) # overrides the datatype of x_d
print(f"Random Tensor: \n {x_rand} \n")
```

```
Ones Tensor:
 tensor([[1, 1],
        [1, 1]])

Random Tensor:
 tensor([[0.9635, 0.9661],
        [0.2867, 0.2272]])
```

With random or constant values:

shape is a tuple of tensor dimensions. In the functions below, it determines the dimensionality of the output tensor.

In [6]:
```
shape = (2,3)  # shape is tuple
rand_tensor = torch.rand(shape)
rand_tensor2 = torch.rand((2,3))
ones_tensor = torch.ones(shape)
zeros_tensor = torch.zeros(shape)

print(f"Random Tensor: \n {rand_tensor} \n")
print(f"Random Tensor2: \n {rand_tensor2} \n")
print(f"Ones Tensor: \n {ones_tensor} \n")
print(f"Zeros Tensor: \n {zeros_tensor}")
```

```
Random Tensor:
 tensor([[0.8504, 0.7291, 0.1134],
        [0.6841, 0.8606, 0.3158]])

Random Tensor2:
 tensor([[0.8384, 0.4924, 0.3183],
        [0.1858, 0.8950, 0.0684]])

Ones Tensor:
 tensor([[1., 1., 1.],
        [1., 1., 1.]])

Zeros Tensor:
 tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

Tensor Attributes

Tensor attributes describe their shape, datatype, and the device on which they are stored.

In [7]:
```
tensor = torch.rand(3,4)

print(f"Shape of tensor: {tensor.shape}")
print(f"Datatype of tensor: {tensor.dtype}")
print(f"Device tensor is stored on: {tensor.device}")
```

```
Shape of tensor: torch.Size([3, 4])
Datatype of tensor: torch.float32
Device tensor is stored on: cpu
```

Tensor Operations

Over 100 tensor operations, including transposing, indexing, slicing, mathematical operations, linear algebra, random sampling, and more are comprehensively described here.

Try out some of the operations from the list. If you're familiar with the NumPy API, you'll find the Tensor API a breeze to use.

Standard numpy-like indexing and slicing:

In [8]:
```
tensorA = torch.ones(4, 4)
print(tensorA, "\n")

tensorA[1,1] = 6
print(tensorA, "\n")

tensorA[1,3] = 4
print(tensorA, "\n")

tensorA[:,2] = 3
tensorA[:,3] = 5
print(tensorA, "\n")

tensorA[:,:] = 2
print(tensorA, "\n")

tensorA[3,:] = 0
print(tensorA)
```

```
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])

tensor([[1., 1., 1., 1.],
        [1., 6., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])

tensor([[1., 1., 1., 1.],
        [1., 6., 1., 4.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])

tensor([[1., 1., 3., 5.],
        [1., 6., 3., 5.],
        [1., 1., 3., 5.],
        [1., 1., 3., 5.]])

tensor([[2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [2., 2., 2., 2.]])

tensor([[2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [0., 0., 0., 0.]])
```

Joining tensors You can use *torch.cat* to concatenate a sequence of tensors along a given dimension.

tensors: A sequence (such as a list or tuple) of PyTorch tensors.

dim: The dimension along which to concatenate them (defaults to 0).

out: An optional output tensor to store the result.

(https://www.codecademy.com/resources/docs/pytorch/tensors/cat),
(https://docs.pytorch.org/docs/stable/generated/torch.stack.html)

See also [torch.stack] (https://pytorch.org/docs/stable/generated/torch.stack.html), another tensor joining operation that is subtly different from *torch.cat*.

In [9]:
```
t1 = torch.cat([tensorA, tensorA, tensorA], dim=0)
print(t1)
print(t1.shape)
```

```
tensor([[2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [0., 0., 0., 0.],
        [2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [0., 0., 0., 0.],
        [2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [2., 2., 2., 2.],
        [0., 0., 0., 0.]])
torch.Size([12, 4])
```

In [10]:
```
t1 = torch.cat([tensorA, tensorA, tensorA], dim=1)
print(t1)
print(t1.shape)
```

```
tensor([[2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 2.],
        [2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 2.],
        [2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 2., 2.],
        [0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.]])
torch.Size([4, 12])
```

In [11]:
```
# Shape (2, 3)
x = torch.tensor([[1, 2, 3],
                  [4, 5, 6]])

# Concatenate along rows (dim=0) -> Result shape becomes (4, 3)
cat_dim0 = torch.cat([x, x], dim=0)
print("Concatenated along dim 0:\n", cat_dim0)

# Concatenate along columns (dim=1) -> Result shape becomes (2, 6)
cat_dim1 = torch.cat([x, x], dim=1)
print("\nConcatenated along dim 1:\n", cat_dim1)
```

```
Concatenated along dim 0:
 tensor([[1, 2, 3],
        [4, 5, 6],
        [1, 2, 3],
        [4, 5, 6]])

Concatenated along dim 1:
 tensor([[1, 2, 3, 1, 2, 3],
        [4, 5, 6, 4, 5, 6]])
```

Multiplying tensors

In [12]:
```
print(tensorA)
# This computes the element-wise product
print(f"tensorA.mul(tensorA) \n {tensorA.mul(tensorA)} \n")
# Alternative syntax:
print(f"tensorA * tensorA \n {tensorA * tensorA}")
```

```
tensor([[2., 2., 2., 2.],
        [2., 2., 2., 2.],
```